App Development Project Report

1. Student Details

Name: Ajay Singh
Roll Number: 24F3004241
Email: 24f3004241@ds.study.iitm.ac.in
About Me: I am a Diploma level student of IIT Madras BS Degree program from Rajasthan. I love building web applications and exploring data-driven technologies. Building applications like Trek Mate allows me to combine my coding skills with real-world management concepts and build useful tools which are easy to use .

2. Project Details

Project Title: Trek Mate (Trekking Management Application)

Problem Statement:

To design and build a web-based application that allows users to explore and book treks, while providing an easy dashboard for guides(staff) to manage their assigned treks, and giving the admin full control over the system's workflow.

Approach:

The application backend is built using Python with the Flask framework and SQLite for database management. For the frontend, I used HTML and Bootstrap 5 along with Jinja2 templates. I implemented a role-based access system (Admin, Staff, and Trekker) so that every user sees only what they are allowed to see. The core logic handles status updates, and booking validations.

3. AI/LLM Declaration

I used Google Gemini to assist me in debugging minor syntax errors, understanding complex Jinja2 loops, and getting suggestions for Bootstrap UI styling. The extent of AI usage is around 15-20%, strictly limited to learning concepts, error tracking, and styling layout. All the core business logic, database relationships, and final code integration were written and implemented manually by me.

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Server-side backend framework for routing and request handling.
SQLite& SQLAlchemy	Relational database to store data and create Object-Relational Models(ORM)
Jinja2	Template engine to dynamically render HTML pages with backend Python data.
Bootstrap 5	Frontend CSS framework used for responsive layout and styling.
Werkzeug Security	Used to securely hash and verify user passwords in the database.

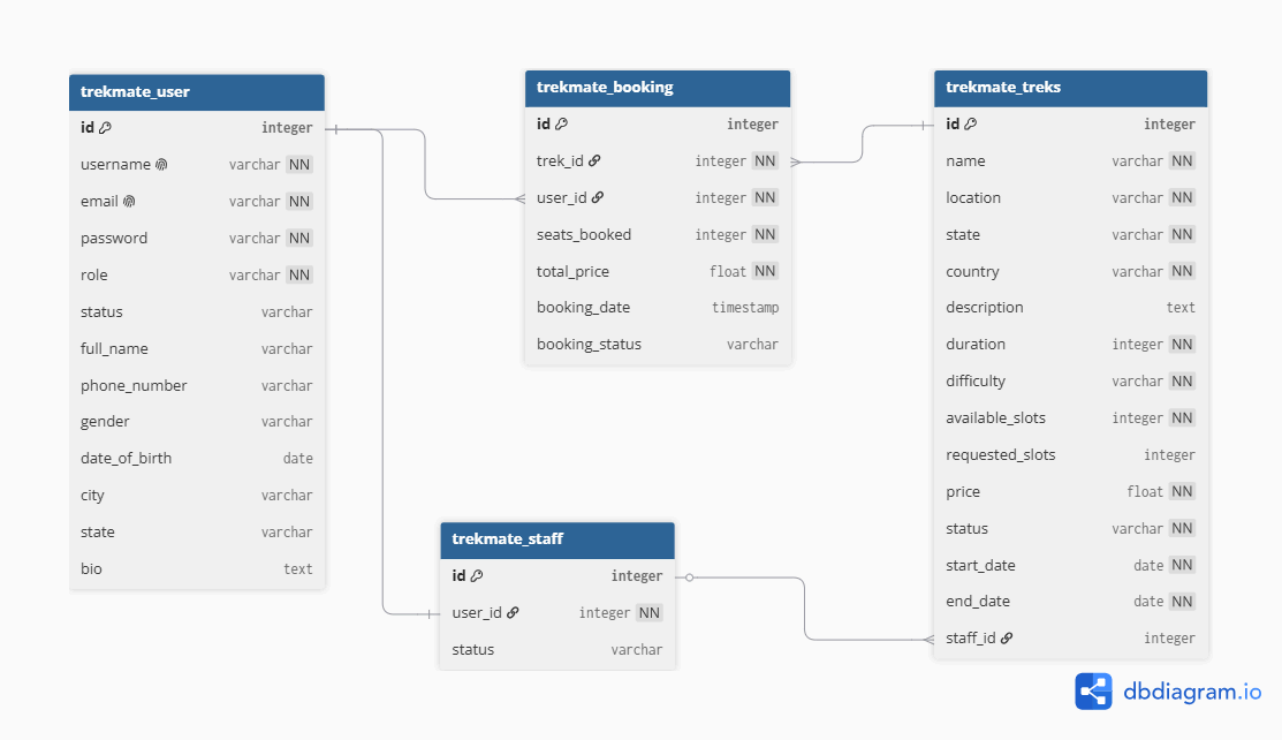
5. Database Schema / ER Diagram

Tables:

1. **User** — stores details of all users (id, username, email, password, role, status, etc).
2. **Staff Profile** — stores specific approval status for guides(id, user_id, status).
3. **Trek** — stores information about available treks (id, name, location, price, slots, start_date, end_date,staff_id).
4. **Booking**— manage user bookings(id, trek_id, user_id, seats_booked, total_price, booking_status).

Relationships:

- **One-to-Many:** User ➔ Booking (One user can make multiple trek bookings).
- **One-to-Many:** Trek ➔ Booking (One trek can have multiple bookings from different users).
- **One-to-Many:** StaffProfile ➔ Trek (One guide/staff can be assigned to multiple treks).
- **One-to-One:** User ➔ StaffProfile (A staff user has exactly one specific staff profile).



6. Application Routing & URL Mapping

Common & User Routes:

Endpoint	Method	Description
/, /home	GET	Landing page of the website.
/register, /login	GET, POST	Register new users and authenticate login sessions.
/explore	GET	View and filter all active upcoming treks.
/book_trek/<trek_id>	GET, POST	Confirm seat selection and book a specific trek.

/my_treks	GET	User dashboard to track upcoming, ongoing, and cancelled bookings.
/logout	GET	Clears the active session and logs the user out.

Admin & Staff Routes:

Endpoint	**Method**	**Description**
/admin_dashboard	GET	Admin panel showing overall system statistics.
/admin/add_trek	GET, POST	Add a new trek into the system and assign a guide.
/admin/manage_treks	GET, POST	Search, edit, or delete existing treks.
/staff_dashboard	GET	Personalized panel for staff to see their assigned treks.
/staff/manage_trek/<trek_id>	GET, POST	Staff can update trek status to ‘Ongoing’ or ‘Completed’, and request slot changes.

7. Architecture and Features

Architecture Overview:
app.py: This is the main backend file. It contains the Flask configuration, SQLAlchemy database models, and all the routing logic.

- templates/ **folder:** Contains all the HTML pages that are rendered using Jinja2 syntax.

Implemented Features:

- **Role-Based Dashboards:** Separate panels and permissions for Admin, Staff, and Trekkers. Admin has full control, Staff can manage assigned treks, and Trekkers can manage their bookings.
- **Smart Booking Engine:** Real-time seat tracking. The system automatically calculates total prices and stops bookings if a trek is ‘Sold Out’.
- **Soft Deletion & Moderation:** Admin can ‘Soft-Delete’ a trek or ‘Block’ a user without breaking the database structure, keeping past historical records intact.

8. Video Presentation

Drive Link: **project_video.mp4**